

Diseño orientado a objetos y arquitectura de software

Breve descripción:

El componente aborda los fundamentos del diseño orientado a objetos y la arquitectura de software, a partir del análisis de requerimientos y la toma de decisiones de diseño. Se presentan conceptos, principios, diagramas, patrones de diseño y tipos de arquitectura, así como arquitecturas modernas y prácticas de desarrollo que permiten estructurar sistemas de software mantenibles, escalables y eficientes.

Tabla de contenido

Introducción	4
1. Análisis y toma de decisiones en el diseño de software	8
1.1. Interpretación del informe de análisis.....	9
1.2. Análisis de requerimientos y toma de decisiones de diseño	11
2. Fundamentos del diseño orientado a objetos.....	13
2.1. Conceptos del diseño orientado a objetos	14
2.2. Principios de calidad en el diseño de software	16
3. Modelado orientado a objetos.....	18
3.1. Diagramas de clases y sus elementos.....	19
3.2. Vistas del sistema	22
4. Patrones de diseño de software	25
4.1. Conceptos y clasificación de patrones de diseño	26
4.2. Patrones creacionales, estructurales y comportamentales.....	28
5. Arquitectura de software	32
5.1. Conceptos.....	33
5.2. Tipos de arquitecturas y cualidades sistémicas	35
5.3. Arquitectura cliente-servidor	37
5.4. Modelo Vista Controlador (MVC)	40

6. Arquitecturas modernas y prácticas de desarrollo	42
6.1. Arquitectura orientada a servicios.....	43
6.2. Arquitecturas basadas en APIs REST y GraphQL.....	45
6.3. Arquitectura de microservicios.....	47
6.4. Arquitectura serverless	49
6.5. DevOps e integración continua.....	51
7. Plataformas tecnológicas y documentación del diseño.....	54
7.1. Plataformas tecnológicas para el desarrollo de software.....	54
7.2. Motores de bases de datos relacionales y no relacionales.....	56
7.3. Documento de diseño de software	58
7.4. Arquitectura candidata.....	60
Síntesis	64
Glosario	66
Referencias bibliográficas	68
Créditos	69

Introducción

El diseño de software es una etapa fundamental en el desarrollo de sistemas informáticos, ya que permite transformar los requerimientos identificados durante el análisis en una estructura organizada que guíe la construcción de la solución tecnológica. En este proceso, las decisiones de diseño influyen directamente en la calidad, el rendimiento, la escalabilidad y la mantenibilidad del sistema.

Uno de los enfoques más utilizados en la ingeniería de software es el diseño orientado a objetos, el cual propone organizar los sistemas a partir de objetos que interactúan entre sí mediante atributos y métodos. Este enfoque permite representar de manera más natural los elementos del mundo real dentro del software y facilita la reutilización, la modularidad y la evolución de las aplicaciones.

Para apoyar este proceso de diseño se emplean herramientas de modelado, como los diagramas de clases, de componentes y de despliegue, que permiten representar la estructura del sistema y las relaciones entre sus elementos. Asimismo, se utilizan patrones de diseño que ofrecen soluciones probadas a problemas comunes en el desarrollo de software.

De igual forma, la arquitectura de software define la organización general del sistema, los componentes que lo conforman y la manera en que interactúan entre sí. En este contexto, han surgido diferentes enfoques arquitectónicos como cliente-servidor, arquitectura orientada a servicios, microservicios y modelos modernos de desarrollo apoyados por prácticas como DevOps e integración continua.

En conjunto, estos elementos permiten construir sistemas de software robustos, escalables y adaptables a los cambios, facilitando su evolución y mantenimiento a lo largo del tiempo.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Diseño orientado a objetos y arquitectura de software



[Enlace de reproducción del video](#)

Transcripción del video. Diseño orientado a objetos y arquitectura de software

El diseño de software es una etapa fundamental en el desarrollo de sistemas informáticos, ya que permite organizar las ideas, planificar la estructura de una aplicación y tomar decisiones que influyen en su funcionamiento, calidad y mantenimiento a lo largo del tiempo.

Transcripción del video. Diseño orientado a objetos y arquitectura de software

En las primeras décadas de la informática, muchos programas se desarrollaban sin una planificación clara. A medida que los sistemas se hicieron más complejos, surgió la necesidad de aplicar métodos y modelos que ayudaran a comprender mejor los requerimientos y a estructurar el software de manera organizada. De esta necesidad nace la ingeniería de software, disciplina que propone enfoques, técnicas y herramientas para analizar, modelar y diseñar sistemas informáticos.

Uno de los enfoques más influyentes es el diseño orientado a objetos, el cual permite representar los sistemas mediante clases, objetos y relaciones. Este modelo facilita la organización del software y permite describir de manera clara los elementos que conforman una aplicación. Para apoyar este proceso se utilizan herramientas de modelado como los diagramas de clases y las diferentes vistas del sistema, que permiten representar la estructura y el comportamiento de una solución tecnológica antes de su implementación.

Con el tiempo, la disciplina ha incorporado patrones de diseño y modelos de arquitectura que ayudan a resolver problemas comunes en la construcción de software. Estos enfoques permiten desarrollar sistemas más organizados, reutilizables y escalables.

Asimismo, el desarrollo de software moderno incorpora arquitecturas y prácticas actuales como los servicios, las APIs, los microservicios, el enfoque serverless y las prácticas de integración continua, que permiten construir

Transcripción del video. Diseño orientado a objetos y arquitectura de software
aplicaciones más flexibles y adaptadas a las necesidades actuales de las organizaciones.

1. Análisis y toma de decisiones en el diseño de software

El análisis en el diseño de software corresponde a la etapa en la que se estudian los requerimientos del sistema con el fin de comprender el problema que se desea resolver y definir una base clara para la construcción de la solución tecnológica. Durante este proceso se revisa la información obtenida en la fase de análisis del sistema, identificando las necesidades de los usuarios, las funciones que debe cumplir la aplicación y las condiciones del entorno en el que operará.

A partir de esta comprensión inicial, el equipo de desarrollo puede transformar los requerimientos en decisiones de diseño que orienten la estructura del software. Estas decisiones influyen en la organización de los componentes del sistema, la forma en que interactúan entre sí y las tecnologías que pueden emplearse para su implementación.

El análisis no se limita únicamente a revisar los requerimientos funcionales del sistema. También implica considerar aspectos relacionados con el desempeño, la seguridad, la escalabilidad y la facilidad de mantenimiento del software, ya que estos factores influyen directamente en la calidad y sostenibilidad de la solución desarrollada.

En este contexto, la toma de decisiones en el diseño de software requiere evaluar diferentes alternativas de solución y seleccionar aquellas que permitan responder de manera adecuada a las necesidades del proyecto. Para ello, es necesario analizar el problema desde una perspectiva técnica y funcional, considerando los recursos disponibles, las restricciones del entorno y los objetivos del sistema.

Un proceso de análisis bien estructurado permite reducir riesgos en el desarrollo, mejorar la organización del sistema y facilitar la implementación de soluciones que sean comprensibles, adaptables y mantenibles a lo largo del tiempo. Por esta razón, el análisis y la toma de decisiones constituyen una base fundamental dentro del proceso de diseño de software.

1.1. Interpretación del informe de análisis

La interpretación del informe de análisis consiste en examinar y comprender la información obtenida durante el levantamiento de requerimientos del sistema. Este informe reúne los resultados del estudio del problema, las necesidades de los usuarios, las funcionalidades esperadas y las restricciones que pueden influir en el desarrollo del software.

Una correcta interpretación permite transformar esta información en una base clara para el diseño del sistema, ya que facilita comprender qué debe hacer la aplicación y bajo qué condiciones deberá operar.

Durante este proceso se analizan distintos elementos del informe con el fin de garantizar que los requerimientos sean comprensibles y útiles para la etapa de diseño.

Entre los aspectos que normalmente se revisan se encuentran los siguientes:

- Requerimientos funcionales del sistema.
- Requerimientos no funcionales relacionados con rendimiento, seguridad o disponibilidad.
- Actores o usuarios que interactúan con el sistema.

- Procesos o flujos de información que debe soportar la aplicación.
- Restricciones técnicas o de negocio que condicionan el desarrollo.

La interpretación también implica revisar la calidad de la información presentada en el informe de análisis. En esta etapa se busca identificar posibles ambigüedades, inconsistencias o vacíos que puedan afectar el diseño del sistema.

Tabla 1. Elementos analizados en la interpretación del informe

Elemento del informe	Qué se analiza	Importancia para el diseño
Requerimientos funcionales.	Funciones que debe cumplir el sistema.	Permiten definir módulos o componentes.
Requerimientos no funcionales.	Calidad del sistema (rendimiento, seguridad y disponibilidad).	Influyen en la arquitectura del software.
Actores del sistema.	Usuarios o sistemas externos que interactúan.	Orientan el diseño de interfaces.
Procesos del sistema.	Flujo de actividades y operaciones.	Definen la lógica de funcionamiento.
Restricciones.	Limitaciones técnicas o de negocio.	Condicionan decisiones tecnológicas.

En síntesis, la interpretación del informe de análisis permite comprender de manera estructurada los requerimientos del sistema y constituye la base para la toma de decisiones durante el diseño del software.

1.2. Análisis de requerimientos y toma de decisiones de diseño

El análisis de requerimientos es el proceso mediante el cual se examina y organiza la información obtenida durante la etapa de análisis del sistema con el propósito de comprender con mayor precisión las necesidades que debe satisfacer el software. A partir de este análisis se identifican las funcionalidades que debe ofrecer el sistema, las restricciones técnicas existentes y los criterios de calidad que orientarán su desarrollo.

Este proceso permite transformar los requerimientos en insumos concretos para el diseño del sistema, facilitando la definición de su estructura, los componentes que lo integran y las tecnologías que se utilizarán en su implementación.

En el análisis de requerimientos se consideran principalmente los siguientes elementos:

- Identificación de funcionalidades que debe cumplir el sistema.
- Determinación de los actores que interactúan con la aplicación.
- Análisis de restricciones técnicas o de negocio.
- Evaluación de requisitos de calidad como rendimiento, seguridad o escalabilidad.
- Priorización de funcionalidades según las necesidades del proyecto.

Una vez analizados los requerimientos, el equipo de desarrollo debe tomar decisiones de diseño que permitan definir la forma en que el sistema será construido. Estas decisiones influyen directamente en la arquitectura del software, la organización de los componentes y las tecnologías seleccionadas.

Tabla 2. Relación entre análisis de requerimientos y decisiones de diseño

Elemento analizado	Pregunta orientadora	Decisión de diseño asociada
Funcionalidades del sistema.	¿Qué debe hacer el sistema?	Definición de módulos o componentes.
Usuarios o actores.	¿Quién interactúa con el sistema?	Diseño de interfaces y flujos de interacción.
Restricciones técnicas.	¿Qué limitaciones existen?	Selección de tecnologías y herramientas.
Requisitos de calidad.	¿Cómo debe comportarse el sistema?	Definición de arquitectura y patrones de diseño.
Prioridad de requerimientos.	¿Qué funcionalidades son más importantes?	Planificación del desarrollo.

En este sentido, el análisis de requerimientos y la toma de decisiones de diseño se encuentran estrechamente relacionados, ya que cada decisión técnica debe basarse en la comprensión clara de las necesidades del sistema. Una adecuada interpretación de los requerimientos permite definir soluciones más coherentes, escalables y mantenibles dentro del proceso de desarrollo de software.

2. Fundamentos del diseño orientado a objetos

El diseño orientado a objetos es un enfoque de desarrollo de software que organiza los sistemas a partir de objetos, los cuales representan entidades del mundo real o conceptos del dominio del problema. Cada objeto integra datos y comportamientos, lo que permite estructurar el software de manera modular, comprensible y fácil de mantener.

Este enfoque se basa en la idea de que un sistema puede modelarse mediante la interacción de diferentes objetos que colaboran entre sí para cumplir determinadas funciones. Cada objeto posee atributos que representan su estado y métodos que definen las acciones que puede realizar.

El diseño orientado a objetos facilita la construcción de sistemas más organizados y adaptables, ya que permite dividir la complejidad del software en componentes más pequeños e independientes. De esta manera, es posible modificar o ampliar ciertas partes del sistema sin afectar de forma significativa el funcionamiento del resto de la aplicación.

Entre las características que distinguen este enfoque se encuentran:

- Representación del sistema mediante clases y objetos.
- Encapsulación de datos y comportamientos en una misma estructura.
- Reutilización de código a través de mecanismos como la herencia y la composición.
- Flexibilidad para extender o modificar funcionalidades del sistema.
- Organización modular que facilita el mantenimiento y evolución del software.

El diseño orientado a objetos se utiliza ampliamente en el desarrollo de aplicaciones modernas, ya que permite construir sistemas más robustos y escalables. Además, este enfoque se integra con herramientas de modelado como los diagramas del lenguaje UML, los cuales permiten representar de manera estructurada los elementos que conforman el sistema antes de su implementación.

2.1. Conceptos del diseño orientado a objetos

El diseño orientado a objetos se fundamenta en un conjunto de conceptos que permiten estructurar el software mediante componentes independientes que interactúan entre sí. Estos conceptos facilitan la organización del código, la reutilización de funcionalidades y la construcción de sistemas más flexibles y mantenibles.

A continuación, se presentan algunos de los conceptos más importantes utilizados en el diseño orientado a objetos.

- **Encapsulamiento:** consiste en agrupar los datos y los métodos que operan sobre ellos dentro de una misma estructura denominada clase. Este principio permite controlar el acceso a la información interna de los objetos, evitando modificaciones directas que puedan afectar el funcionamiento del sistema.
- **Herencia:** permite crear nuevas clases a partir de otras ya existentes. La clase derivada hereda atributos y métodos de la clase base, lo que facilita la reutilización de código y la extensión de funcionalidades sin duplicar estructuras.

- **Polimorfismo:** permite que diferentes objetos respondan de manera distinta a un mismo mensaje o método. Esto significa que una misma operación puede ejecutarse de formas diferentes dependiendo del tipo de objeto que la utilice.
- **Composición:** se refiere a la construcción de objetos complejos a partir de otros objetos más simples. Este mecanismo permite modelar relaciones de “parte de” dentro del sistema, promoviendo estructuras más organizadas y reutilizables.
- **Interfaces:** se definen un conjunto de métodos que una clase debe implementar. No describen cómo se ejecuta una acción, sino qué operaciones deben estar disponibles, lo que permite establecer contratos de comportamiento entre diferentes componentes del sistema.
- **Cohesión:** se relaciona con el grado en que los elementos de una clase o módulo están enfocados en una única responsabilidad. Una alta cohesión indica que los componentes cumplen funciones claramente definidas dentro del sistema.
- **Acoplamiento:** describe el nivel de dependencia entre diferentes clases o módulos. En un diseño adecuado se busca mantener un bajo acoplamiento, de manera que los cambios en un componente no afecten significativamente a los demás.

Estos conceptos constituyen la base del diseño orientado a objetos y permiten construir sistemas organizados, flexibles y fáciles de evolucionar. En el siguiente apartado se presentan los principios que orientan la calidad del diseño de software.

2.2. Principios de calidad en el diseño de software

El diseño de software no solo busca estructurar correctamente los componentes de un sistema, sino también garantizar que el producto resultante cumpla con criterios de calidad que faciliten su evolución, mantenimiento y adaptación a diferentes contextos tecnológicos. Para ello, el diseño orientado a objetos se apoya en una serie de principios que orientan la construcción de sistemas eficientes, robustos y sostenibles en el tiempo.

Estos principios permiten evaluar si una solución técnica es adecuada y si el sistema podrá responder a cambios futuros sin generar altos costos de modificación.

A continuación, se presentan algunos de los principios de calidad más relevantes en el diseño de software.

- **Adaptabilidad:** se refiere a la capacidad del sistema para ajustarse a cambios en el entorno, en los requerimientos o en las tecnologías utilizadas. Un diseño adaptable facilita la incorporación de nuevas funcionalidades sin alterar significativamente la estructura existente.
- **Extensibilidad:** permite ampliar las capacidades del sistema mediante la adición de nuevos módulos o componentes. Un sistema extensible está preparado para crecer sin necesidad de modificar el código base de manera profunda.
- **Mantenibilidad:** describe la facilidad con la que el software puede ser corregido, actualizado o mejorado. Un diseño claro, modular y bien documentado facilita la identificación de errores y la implementación de mejoras.

- **Reusabilidad:** implica que los componentes del sistema puedan utilizarse en otros proyectos o en diferentes partes del mismo sistema. Esto reduce el tiempo de desarrollo y mejora la consistencia del software.
- **Desempeño:** se relaciona con la eficiencia con la que el sistema utiliza los recursos disponibles, como memoria, procesamiento y almacenamiento. Un diseño adecuado busca optimizar estos recursos para garantizar un funcionamiento rápido y estable.
- **Escalabilidad:** hace referencia a la capacidad del sistema para soportar un aumento en la carga de trabajo, ya sea en número de usuarios, volumen de datos o transacciones. Un sistema escalable puede crecer sin comprometer su rendimiento.
- **Confiabilidad:** indica el grado en que el sistema funciona correctamente durante un periodo determinado sin presentar fallos. Un diseño confiable incorpora mecanismos que permiten prevenir errores y garantizar la estabilidad del sistema.
- **Eficiencia:** se relaciona con la utilización adecuada de los recursos del sistema para cumplir con los objetivos funcionales sin generar desperdicio de capacidad computacional.

La aplicación de estos principios permite desarrollar sistemas más robustos, sostenibles y preparados para evolucionar a lo largo del tiempo. Estos criterios se complementan con técnicas de modelado que permiten representar la estructura del sistema antes de su implementación.

3. Modelado orientado a objetos

El modelado orientado a objetos es una técnica utilizada para representar de manera estructurada los elementos que componen un sistema de software y las relaciones que existen entre ellos. Este proceso permite visualizar la organización del sistema antes de su implementación, facilitando la comprensión del diseño y la comunicación entre los diferentes miembros del equipo de desarrollo.

A través del modelado, los diseñadores pueden describir cómo se estructuran las clases, cómo interactúan los componentes y cómo se distribuyen los elementos dentro de la arquitectura del sistema. Esta representación ayuda a identificar posibles problemas en etapas tempranas del desarrollo y permite realizar ajustes antes de iniciar la fase de programación.

El modelado orientado a objetos se apoya en diagramas que permiten representar diferentes perspectivas del sistema. Estos diagramas forman parte del Lenguaje Unificado de Modelado (UML), ampliamente utilizado en la ingeniería de software para documentar y comunicar decisiones de diseño.

Entre los principales objetivos del modelado se encuentran los siguientes:

- Representar la estructura del sistema de manera clara y organizada.
- Identificar las clases, sus atributos y métodos.
- Definir las relaciones entre los diferentes elementos del sistema.
- Visualizar cómo se distribuyen los componentes en la arquitectura.
- Facilitar la documentación del diseño de software.

El uso de modelos permite reducir la complejidad del sistema, ya que proporciona una representación conceptual que sirve como guía para el desarrollo. De

esta manera, el equipo puede analizar el comportamiento del sistema, evaluar alternativas de diseño y tomar decisiones informadas antes de escribir código.

3.1. Diagramas de clases y sus elementos

El diagrama de clases es uno de los modelos más utilizados en el diseño orientado a objetos, ya que permite representar la estructura estática de un sistema. En este tipo de diagrama se describen las clases que componen el sistema, junto con sus atributos, métodos y las relaciones que existen entre ellas.

Este tipo de representación facilita comprender cómo se organiza la información dentro del sistema y cómo interactúan los diferentes elementos que lo componen. Por esta razón, los diagramas de clases se utilizan con frecuencia durante la etapa de diseño para planificar la estructura del software antes de iniciar la programación.

Una clase representa una plantilla o modelo que define las características y comportamientos de un conjunto de objetos. Cada clase puede incluir atributos, que describen las propiedades del objeto, y métodos, que representan las acciones que puede realizar.

Los diagramas de clases están compuestos por diferentes elementos que permiten representar la estructura del sistema de manera clara.

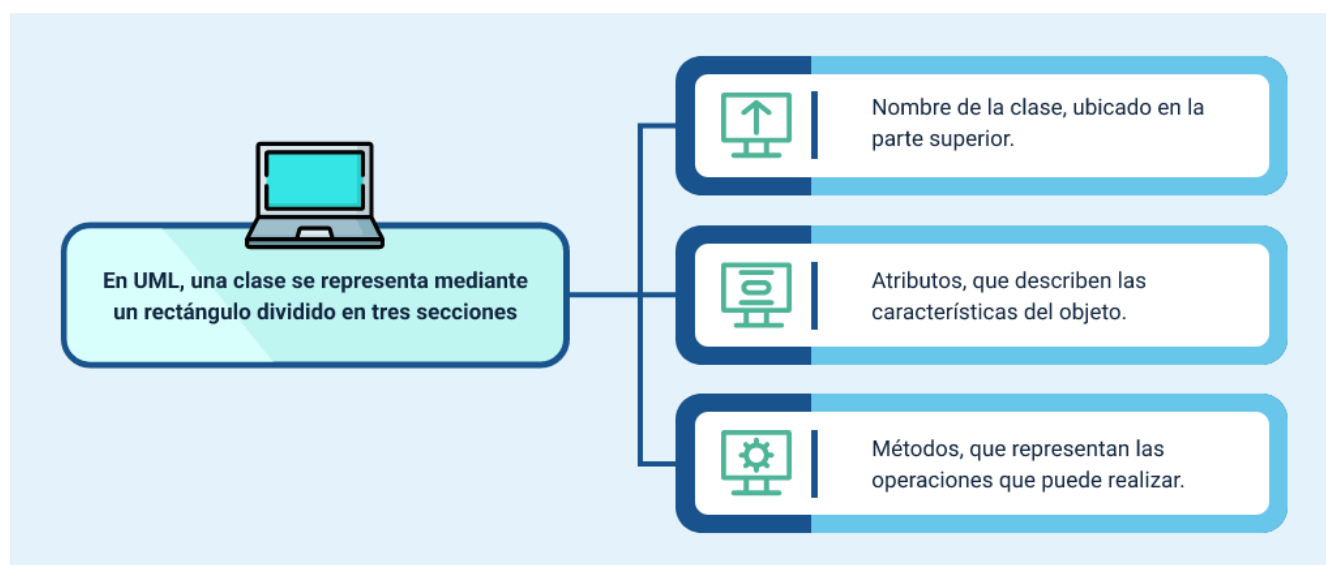
Tabla 3. Elementos principales de un diagrama de clases

Elemento	Descripción	Ejemplo
Clase.	Representa una entidad del sistema que agrupa atributos y métodos.	Clase Usuario

Elemento	Descripción	Ejemplo
Atributos.	Propiedades que describen las características de la clase.	nombre, correo, contraseña
Métodos.	Funciones o comportamientos que puede realizar la clase.	iniciarSesion(), registrarUsuario()
Asociación.	Relación entre dos clases que indica interacción o dependencia.	Usuario realiza Pedido
Herencia.	Relación donde una clase hereda características de otra.	Empleado hereda de Persona
Composición.	Relación fuerte donde un objeto depende completamente de otro.	Pedido contiene DetallePedido
Agregación.	Relación donde una clase agrupa a otra, pero ambas pueden existir de forma independiente.	Curso contiene Estudiantes

Representación básica de una clase:

Figura 1. Clase en UML



A continuación, se presenta un ejemplo conceptual simplificado de la estructura de una clase:

Tabla 4. Ejemplo conceptual de una clase

Clase	Atributos	Métodos
Usuario	idUsuario nombre correo contraseña	registrar() iniciarSesion() cerrarSesion()

Esta estructura permite representar de forma organizada la información que pertenece a cada clase dentro del sistema.

Además de representar clases individuales, los diagramas de clases permiten describir las relaciones entre los diferentes elementos del sistema. Estas relaciones ayudan a comprender cómo interactúan las clases dentro de la aplicación.

Las relaciones más comunes son:

- **Asociación:** indica que dos clases se relacionan entre sí.
- **Herencia:** permite reutilizar características de una clase base en otra clase derivada.
- **Agregación:** representa una relación donde un objeto contiene a otro, pero ambos pueden existir de forma independiente.
- **Composición:** indica una relación fuerte donde un objeto no puede existir sin el otro.

Para comprender mejor estas relaciones, se puede considerar un ejemplo conceptual aplicado a un sistema de gestión de biblioteca. En este sistema intervienen las clases Libro, Usuario y Préstamo.

Un Usuario puede solicitar uno o varios Préstamos, mientras que cada Préstamo se asocia con un Libro específico. Además, el sistema registra información relevante como la fecha de préstamo y la fecha de devolución.

El diagrama de clases permite representar estas relaciones y comprender la estructura del sistema antes de implementar la solución.

En síntesis, los diagramas de clases constituyen una herramienta fundamental para el diseño de software, ya que permiten organizar la estructura del sistema, identificar las relaciones entre los componentes y facilitar la documentación del diseño dentro del proceso de desarrollo orientado a objetos.

3.2. Vistas del sistema

En el proceso de diseño de software, los sistemas suelen ser complejos y están compuestos por múltiples componentes que interactúan entre sí. Para facilitar su comprensión y organización, se utilizan vistas del sistema, las cuales permiten representar el sistema desde diferentes perspectivas.

Cada vista describe un aspecto particular del sistema, lo que permite analizarlo de manera estructurada sin necesidad de prestar atención a todos sus elementos al mismo tiempo. De esta forma, las vistas ayudan a comprender cómo funciona el sistema, cómo se organiza y cómo interactúan sus componentes.

El uso de vistas resulta especialmente útil en proyectos de desarrollo de software, ya que diferentes actores como analistas, diseñadores, desarrolladores o arquitectos, pueden enfocarse en la información que resulta más relevante para su rol.

En el diseño de software existen diferentes tipos de vistas que permiten representar diversos aspectos del sistema.

Tabla 5. Principales vistas del sistema

Vista	Qué describe	Propósito en el diseño
Vista lógica.	Representa la estructura del sistema y la organización de las clases y objetos.	Comprender la organización funcional del sistema.
Vista de procesos.	Describe la interacción y comunicación entre procesos o componentes.	Analizar el comportamiento dinámico del sistema.
Vista de desarrollo.	Muestra cómo se organizan los componentes del software dentro del entorno de desarrollo.	Facilitar la organización del código y los módulos.
Vista física o de despliegue.	Representa cómo se distribuyen los componentes del sistema en el hardware o infraestructura.	Planificar la implementación del sistema.

Cada una de estas vistas permite analizar el sistema desde una perspectiva diferente, lo que facilita la toma de decisiones durante el diseño y la planificación del software.

Relación entre las vistas del sistema

Las vistas no se utilizan de forma aislada, sino que se complementan entre sí para ofrecer una comprensión integral del sistema. Mientras una vista permite analizar la

estructura lógica del software, otra puede mostrar su comportamiento o su distribución en la infraestructura tecnológica.

Por ejemplo, la vista lógica permite identificar las clases y relaciones del sistema mediante diagramas de clases, mientras que la vista de despliegue permite comprender cómo se distribuyen los componentes en servidores o dispositivos dentro de una infraestructura tecnológica.

Esta combinación de perspectivas permite a los equipos de desarrollo diseñar sistemas más organizados, escalables y fáciles de mantener.

En síntesis, las vistas del sistema constituyen una herramienta fundamental en el modelado de software, ya que permiten representar diferentes aspectos del sistema de manera estructurada, facilitando la comunicación entre los miembros del equipo y apoyando la toma de decisiones durante el proceso de diseño.

4. Patrones de diseño de software

En el desarrollo de software, es común enfrentar problemas similares durante el proceso de diseño de los sistemas. Para abordar estas situaciones de manera eficiente, se utilizan patrones de diseño, los cuales representan soluciones probadas y reutilizables para resolver problemas recurrentes dentro del diseño de software.

Un patrón de diseño no corresponde a un código específico ni a una implementación exacta, sino a una estructura conceptual que orienta la forma en que se organizan los componentes del sistema para resolver un problema determinado. De esta manera, los patrones proporcionan guías que ayudan a los desarrolladores a diseñar soluciones más organizadas, flexibles y fáciles de mantener.

El uso de patrones de diseño permite aprovechar el conocimiento acumulado en la ingeniería de software, ya que estos patrones han sido identificados y documentados a partir de experiencias reales en el desarrollo de sistemas. Por esta razón, constituyen una herramienta importante para mejorar la calidad del diseño y facilitar la comunicación entre los miembros del equipo de desarrollo.

Entre los principales beneficios de los patrones de diseño se destacan:

- Facilitan la reutilización de soluciones a problemas comunes.
- Promueven diseños más organizados y estructurados.
- Mejoran la mantenibilidad y la escalabilidad del software.
- Favorecen la comunicación entre desarrolladores mediante un lenguaje común de diseño.

En el contexto del diseño orientado a objetos, los patrones de diseño ayudan a definir cómo deben relacionarse las clases y los objetos para resolver situaciones

frecuentes dentro de un sistema. Esto permite construir aplicaciones más flexibles y adaptables a cambios futuros.

En síntesis, los patrones de diseño constituyen una herramienta fundamental dentro del diseño de software, ya que proporcionan modelos de solución reutilizables que permiten estructurar sistemas de manera más eficiente, mejorar la calidad del código y apoyar el proceso de desarrollo de aplicaciones orientadas a objetos.

4.1. Conceptos y clasificación de patrones de diseño

Los patrones de diseño son soluciones generales y reutilizables que se aplican a problemas recurrentes en el diseño de software. No representan un algoritmo específico ni una implementación exacta, sino una forma estructurada de organizar clases y objetos para resolver situaciones comunes dentro de un sistema.

En el contexto del desarrollo orientado a objetos, los patrones ayudan a mejorar la organización del software, ya que proponen formas adecuadas de distribuir responsabilidades entre los componentes del sistema. De esta manera, permiten construir aplicaciones más flexibles, reutilizables y fáciles de mantener.

Un patrón de diseño generalmente describe varios elementos que orientan su aplicación dentro del sistema:

- Nombre del patrón, que permite identificar la solución propuesta.
- Problema, que describe la situación o necesidad que se desea resolver.
- Solución, que explica cómo deben organizarse los elementos del sistema para resolver el problema.

- Consecuencias, que muestran los beneficios y posibles limitaciones del patrón.

Estos elementos permiten comprender cuándo y cómo aplicar un patrón dentro del proceso de diseño.

Los patrones de diseño suelen clasificarse según el tipo de problema que ayudan a resolver dentro de la estructura del software. Esta clasificación facilita su estudio y aplicación durante el diseño del sistema.

Tabla 6. Clasificación general de los patrones de diseño

Tipo de patrón	Qué aborda	En qué se enfoca
Patrones creacionales.	Gestionan la forma en que se crean los objetos dentro del sistema.	Facilitar la creación flexible de objetos.
Patrones estructurales.	Definen cómo se relacionan y organizan las clases y objetos.	Mejorar la estructura del sistema.
Patrones comportamentales.	Describen la comunicación y distribución de responsabilidades entre objetos.	Optimizar la interacción entre componentes.

Esta clasificación permite comprender que los patrones no se aplican todos para el mismo propósito, sino que cada grupo responde a necesidades diferentes dentro del diseño de software.

En síntesis, comprender el concepto y la clasificación de los patrones de diseño permite seleccionar soluciones adecuadas para organizar el sistema de manera eficiente, mejorar la reutilización del código y facilitar la evolución del software a lo largo del tiempo.

4.2. Patrones creacionales, estructurales y comportamentales

Los patrones de diseño pueden agruparse en tres grandes categorías según el tipo de problema que buscan resolver dentro del sistema. Estas categorías permiten comprender cómo se organiza el software, cómo se crean los objetos y cómo interactúan entre sí los diferentes componentes de una aplicación.

Cada grupo de patrones aborda un aspecto específico del diseño orientado a objetos, lo que facilita la construcción de sistemas más estructurados, flexibles y mantenibles.

- a. **Patrones creacionales:** se enfocan en la forma en que se crean los objetos dentro del sistema. Su propósito es controlar el proceso de instanciación, evitando dependencias rígidas entre las clases y permitiendo que el sistema sea más flexible frente a cambios.

Estos patrones ayudan a separar la lógica de creación de objetos de la lógica principal del sistema, lo que facilita la reutilización del código y la adaptación a diferentes contextos de ejecución.

Tabla 7. Ejemplos de patrones creacionales

Patrón	Descripción	Uso principal
Singleton	Garantiza que una clase tenga una única instancia durante la ejecución del sistema.	Controlar el acceso a recursos compartidos.
Factory Method	Define una interfaz para crear objetos sin especificar su clase concreta.	Delegar la creación de objetos.

Patrón	Descripción	Uso principal
Abstract Factory	Permite crear familias de objetos relacionados sin especificar sus clases concretas.	Construcción de sistemas configurables.
Builder	Separa la construcción de un objeto complejo de su representación final.	Crear objetos paso a paso.

Para complementar la información sobre patrones creacionales, se recomienda acceder al siguiente video: <https://www.youtube.com/watch?v= KZkbLOMMbQ>

b. Patrones estructurales: se centran en la organización de las clases y objetos dentro del sistema. Su objetivo es facilitar la forma en que los componentes se relacionan entre sí para construir estructuras más claras y eficientes.

Estos patrones permiten combinar diferentes clases para formar estructuras más complejas sin perder flexibilidad ni aumentar innecesariamente el acoplamiento entre los componentes del sistema.

Tabla 8. Ejemplos de patrones estructurales

Patrón	Descripción	Uso principal
Adapter	Permite que clases con interfaces incompatibles trabajen juntas.	Integrar componentes existentes.
Facade	Proporciona una interfaz simplificada para un conjunto de clases complejas.	Simplificar el acceso a subsistemas.

Patrón	Descripción	Uso principal
Composite	Permite tratar objetos individuales y composiciones de objetos de forma uniforme.	Representar estructuras jerárquicas.
Decorator	Añade funcionalidades a un objeto de manera dinámica.	Extender comportamiento sin modificar la clase.

- c. Patrones comportamentales:** se enfocan en la comunicación entre objetos y en la forma en que se distribuyen las responsabilidades dentro del sistema. Estos patrones permiten definir cómo interactúan los componentes para ejecutar tareas de manera coordinada.

Su aplicación facilita la organización de la lógica del sistema y permite reducir el acoplamiento entre los diferentes elementos que participan en un proceso.

Tabla 9. Ejemplos de patrones comportamentales

Patrón	Descripción	Uso principal
Observer	Permite que varios objetos sean notificados cuando cambia el estado de otro objeto.	Sistemas de eventos o notificaciones.
Strategy	Define diferentes algoritmos intercambiables para realizar una tarea.	Seleccionar comportamientos dinámicamente.
Command	Encapsula una solicitud como un objeto independiente.	Desacoplar quien solicita la acción de quien la ejecuta.

Patrón	Descripción	Uso principal
State	Permite que un objeto cambie su comportamiento según su estado interno.	Modelar comportamientos dependientes del estado.

En síntesis, los patrones creacionales, estructurales y comportamentales ofrecen soluciones organizadas para diferentes aspectos del diseño de software. Su aplicación permite desarrollar sistemas más flexibles, reutilizables y fáciles de mantener dentro de los procesos de desarrollo orientados a objetos.

5. Arquitectura de software

La arquitectura de software constituye la estructura fundamental que define cómo se organizan los componentes de un sistema, cómo interactúan entre sí y cuáles son los principios que guían su construcción y evolución. En esta etapa del diseño se establecen las decisiones estructurales que permiten garantizar que el sistema cumpla con los requerimientos funcionales y no funcionales definidos durante el análisis.

Desde una perspectiva conceptual, la arquitectura describe la organización global del sistema, incluyendo sus módulos principales, las relaciones entre ellos, los mecanismos de comunicación y las tecnologías que soportan su funcionamiento. Estas decisiones permiten establecer una base sólida que orienta el desarrollo, facilita la comprensión del sistema y reduce riesgos asociados a cambios futuros.

El concepto de arquitectura de software comenzó a consolidarse como disciplina dentro de la ingeniería de software durante la década de 1990, cuando se reconoció la necesidad de estructurar los sistemas complejos mediante modelos y principios arquitectónicos. Autores como Mary Shaw y David Garlan contribuyeron significativamente al desarrollo del concepto moderno de arquitectura, destacando la importancia de describir los sistemas a través de componentes, conectores y configuraciones estructurales.

De acuerdo con lo planteado en el libro *Software Architecture: Perspectives on an Emerging Discipline*, la arquitectura permite comprender el sistema desde una visión de alto nivel que facilita la toma de decisiones técnicas, la comunicación entre los miembros del equipo y la planificación del crecimiento del sistema.

En el proceso de desarrollo de software, definir la arquitectura implica analizar diversos aspectos del sistema, entre los que se destacan:

- La organización de los componentes del sistema.
- Los mecanismos de interacción entre módulos o servicios.
- La distribución de responsabilidades entre los diferentes elementos.
- Las tecnologías que soportarán la implementación.
- Las cualidades de calidad que se desean garantizar, como escalabilidad, seguridad o mantenibilidad.

Una arquitectura bien definida permite reducir la complejidad del sistema y proporciona una guía clara para el desarrollo del software. Además, facilita la evolución del sistema a lo largo del tiempo, ya que establece principios estructurales que permiten incorporar nuevas funcionalidades sin afectar significativamente los componentes existentes.

En síntesis, la arquitectura de software representa el marco estructural que orienta el diseño y la construcción de sistemas informáticos, permitiendo organizar sus componentes, definir sus interacciones y garantizar que el sistema cumpla con los objetivos técnicos y funcionales del proyecto.

5.1. Conceptos

La arquitectura de software corresponde a la organización estructural de un sistema, donde se definen los componentes que lo conforman, las relaciones entre ellos y los principios que orientan su diseño y evolución. Esta arquitectura actúa como un

plano que guía el desarrollo del sistema, permitiendo comprender cómo se distribuyen sus funcionalidades y cómo interactúan los diferentes elementos que lo integran.

Desde una perspectiva conceptual, la arquitectura permite establecer una visión global del sistema antes de su implementación. Esto facilita tomar decisiones relacionadas con la organización de los componentes, la distribución de responsabilidades, la comunicación entre módulos y la selección de tecnologías que soportarán el funcionamiento del software.

Según Len Bass, la arquitectura de software describe las estructuras de un sistema, los elementos que las componen, las propiedades visibles de dichos elementos y las relaciones que existen entre ellos. Esta definición resalta que la arquitectura no solo describe componentes, sino también la forma en que estos colaboran para cumplir los objetivos del sistema.

En el proceso de desarrollo, la arquitectura cumple varias funciones importantes:

- Proporciona una estructura organizativa para el sistema.
- Facilita la comunicación entre los miembros del equipo de desarrollo.
- Permite analizar decisiones técnicas antes de la implementación.
- Sirve como base para la documentación y mantenimiento del software.

De esta manera, la arquitectura se convierte en un elemento fundamental para asegurar que el sistema sea comprensible, escalable y fácil de mantener a lo largo del tiempo.

Tabla 10. Elementos conceptuales de la arquitectura de software

Elemento	Descripción
Componentes.	Partes del sistema que realizan funciones específicas.
Conectores.	Mecanismos que permiten la comunicación entre componentes.
Configuración.	Forma en que se organizan y relacionan los componentes.
Restricciones.	Reglas o decisiones que condicionan la estructura del sistema.

En síntesis, los conceptos de arquitectura de software permiten comprender cómo se organiza un sistema a nivel estructural, proporcionando una base conceptual que orienta el diseño, la implementación y la evolución de las aplicaciones informáticas.

Para complementar la información sobre la arquitectura de software, se recomienda acceder al siguiente video:

<https://www.youtube.com/watch?v=kiV7aeJCqUQ>

5.2. Tipos de arquitecturas y cualidades sistémicas

En el diseño de software, la arquitectura puede organizarse de diferentes maneras según las necesidades del sistema, el tipo de aplicación y el entorno en el que se implementará. Estas organizaciones estructurales se conocen como tipos de arquitectura, y cada una define una forma particular de distribuir los componentes, las responsabilidades y la comunicación dentro del sistema.

La elección de una arquitectura adecuada influye directamente en el rendimiento, la escalabilidad y la facilidad de mantenimiento del software. Por esta

razón, durante el proceso de diseño es necesario evaluar las características del proyecto antes de seleccionar una arquitectura específica.

A continuación, se presentan algunos tipos de arquitecturas utilizados con frecuencia en el desarrollo de software.

Tabla 11. Tipos de arquitecturas de software

Tipo de arquitectura	Descripción	Característica principal
Arquitectura monolítica.	El sistema se desarrolla como una única aplicación donde todos los componentes están integrados.	Simplicidad en el desarrollo inicial.
Arquitectura cliente-servidor.	Divide el sistema entre clientes que realizan solicitudes y servidores que procesan la información.	Distribución de responsabilidades.
Arquitectura en capas.	Organiza el sistema en diferentes niveles funcionales, como presentación, lógica de negocio y datos.	Separación de responsabilidades.
Arquitectura orientada a servicios.	El sistema se compone de servicios independientes que se comunican entre sí.	Integración y reutilización de servicios.
Arquitectura de microservicios.	El sistema se divide en pequeños servicios autónomos que pueden desarrollarse y desplegarse de forma independiente.	Alta escalabilidad y flexibilidad.

Además de los tipos de arquitectura, en el diseño de software también se consideran las cualidades sistémicas, conocidas también como atributos de calidad. Estas cualidades permiten evaluar qué tan adecuado es el diseño de un sistema frente a diferentes condiciones de uso.

Las cualidades sistémicas representan propiedades que influyen en el comportamiento, la eficiencia y la capacidad de evolución del software.

Tabla 12. Principales cualidades sistémicas

Cualidad	Descripción	Importancia en el sistema
Escalabilidad.	Capacidad del sistema para crecer y soportar mayor carga de trabajo.	Permite ampliar el sistema sin afectar su funcionamiento.
Mantenibilidad.	Facilidad para modificar o actualizar el sistema.	Reduce costos de mantenimiento.
Confiabilidad.	Capacidad del sistema para funcionar correctamente durante el tiempo esperado.	Garantiza estabilidad en la operación.
Desempeño.	Capacidad para responder de manera eficiente a las solicitudes.	Mejora la experiencia del usuario.
Seguridad.	Protección de la información y de los recursos del sistema.	Previene accesos no autorizados.

En conjunto, los tipos de arquitectura y las cualidades sistémicas permiten orientar las decisiones de diseño, ya que ayudan a seleccionar la estructura más adecuada para cada proyecto y a garantizar que el sistema cumpla con los requisitos técnicos y operativos establecidos.

5.3. Arquitectura cliente-servidor

La arquitectura cliente-servidor es uno de los modelos más utilizados en el desarrollo de sistemas informáticos y aplicaciones web. En este modelo, el sistema se divide en dos tipos principales de componentes: clientes y servidores.

El cliente es la aplicación o dispositivo que realiza solicitudes de información o servicios, mientras que el servidor es el sistema encargado de procesar esas solicitudes y enviar una respuesta.

Este modelo permite distribuir las responsabilidades dentro del sistema, lo que facilita la organización del software y mejora la eficiencia en la gestión de recursos.

En muchos sistemas actuales, el cliente suele ser una aplicación instalada en un dispositivo o un navegador web, mientras que el servidor se ejecuta en una infraestructura centralizada donde se almacenan los datos y la lógica principal del sistema.

Para comprender mejor este modelo, es importante identificar los elementos que participan en la comunicación entre cliente y servidor.

Tabla 13. Componentes de la arquitectura cliente-servidor

Componente	Descripción	Ejemplo
Cliente.	Aplicación que solicita información o servicios al servidor.	Navegador web.
Servidor.	Sistema que procesa las solicitudes y envía respuestas.	Servidor web.
Red.	Medio de comunicación entre cliente y servidor.	Internet.
Base de datos.	Sistema donde se almacenan los datos del sistema.	Sistema gestor de bases de datos.

El funcionamiento de este modelo se basa en un proceso de solicitud y respuesta.

- El cliente envía una solicitud al servidor.
- El servidor recibe y procesa la solicitud.
- El servidor accede a los recursos necesarios, como bases de datos o servicios.
- El servidor envía una respuesta al cliente.
- El cliente presenta la información al usuario.

Este mecanismo permite que múltiples clientes puedan comunicarse con un mismo servidor, lo que hace posible el funcionamiento de servicios como páginas web, aplicaciones móviles y sistemas empresariales.

Un ejemplo sencillo de arquitectura cliente-servidor puede presentarse en una aplicación web de comercio electrónico, como la utilizada por plataformas de compras en línea como Amazon.

En este tipo de aplicaciones, el navegador web funciona como cliente, mientras que el sistema que procesa la información y gestiona los datos funciona como servidor.

Ejemplo de funcionamiento de un usuario que desea buscar unos audífonos en la página web.

- **Solicitud del cliente:** el usuario escribe “audífonos” en la barra de búsqueda del navegador.
- **Envío de la solicitud:** el navegador envía una solicitud al servidor web para obtener los productos relacionados con esa búsqueda.
- **Procesamiento en el servidor:** el servidor recibe la solicitud, consulta la base de datos donde están almacenados los productos y obtiene la información correspondiente.

- **Respuesta del servidor:** el servidor envía al navegador la lista de productos encontrados.
- **Visualización en el cliente:** el navegador presenta los resultados al usuario en forma de lista con imágenes, precios y descripciones.

Este proceso explica el funcionamiento de la arquitectura cliente-servidor: el cliente solicita información, el servidor la procesa y responde, y el usuario puede interactuar con el sistema a través de esa comunicación continua entre ambos componentes.

5.4. Modelo Vista Controlador (MVC)

El Modelo Vista Controlador (MVC) es un patrón de arquitectura de software que organiza una aplicación en tres componentes principales con el fin de separar la lógica del sistema, la presentación de la información y la gestión de las interacciones del usuario. Esta separación facilita el mantenimiento del sistema, mejora la organización del código y permite desarrollar aplicaciones más escalables.

El modelo divide la aplicación en las siguientes partes:

- **Modelo (model):** representa los datos del sistema y la lógica de negocio. Se encarga de gestionar la información, realizar cálculos, aplicar reglas del sistema y comunicarse con la base de datos. El modelo no depende de la interfaz de usuario.
- **Vista (view):** es la interfaz que muestra la información al usuario. Su función es presentar los datos provenientes del modelo de manera

comprensible, por ejemplo mediante páginas web, formularios o paneles de visualización.

- **Controlador (controller):** actúa como intermediario entre el modelo y la vista. Recibe las acciones del usuario (clics, formularios y solicitudes), interpreta estas solicitudes y decide qué acciones debe ejecutar el modelo y qué vista se debe mostrar.

A continuación, un ejemplo sencillo en una aplicación web:

En una plataforma de reservas de hoteles:

- **Modelo:** gestiona la información de los hoteles, habitaciones disponibles, precios y reservas almacenadas en la base de datos.
- **Vista:** presenta al usuario las páginas web donde puede buscar hoteles, revisar habitaciones disponibles y realizar la reserva.
- **Controlador:** recibe la solicitud del usuario cuando busca un hotel, consulta la información en el modelo y envía los resultados a la vista para que se presenten en la página.

De esta manera, el modelo se encarga de los datos, la vista presenta la información y el controlador coordina la interacción entre ambos, permitiendo que el sistema funcione de forma organizada y eficiente.

6. Arquitecturas modernas y prácticas de desarrollo

El desarrollo de software ha evolucionado significativamente debido al crecimiento de Internet, las aplicaciones distribuidas y la necesidad de construir sistemas cada vez más escalables, flexibles y fáciles de mantener. En este contexto, han surgido nuevas arquitecturas y prácticas de desarrollo que permiten diseñar soluciones tecnológicas más eficientes y adaptadas a entornos dinámicos.

Las arquitecturas modernas buscan resolver problemas asociados con la complejidad de los sistemas, la integración entre múltiples servicios y la necesidad de desplegar aplicaciones de forma rápida y segura. Para ello, se emplean enfoques que promueven la modularidad, la comunicación entre servicios y la automatización de procesos de desarrollo.

Entre estos enfoques se encuentran la arquitectura orientada a servicios, los microservicios, las arquitecturas serverless y las prácticas asociadas a DevOps y la integración continua. Estos modelos permiten que las aplicaciones se construyan mediante componentes independientes que pueden desarrollarse, actualizarse y desplegarse de manera autónoma.

Asimismo, el uso de interfaces de comunicación como REST y GraphQL facilita el intercambio de información entre sistemas, permitiendo que diferentes aplicaciones y servicios se integren de forma eficiente dentro de una misma plataforma tecnológica.

En conjunto, estas arquitecturas y prácticas contribuyen a crear sistemas más flexibles, escalables y preparados para responder a las necesidades cambiantes de las organizaciones y de los usuarios.

6.1. Arquitectura orientada a servicios

La arquitectura orientada a servicios (SOA, Service-Oriented Architecture) es un enfoque de diseño de software que organiza las funcionalidades de un sistema en servicios independientes que pueden comunicarse entre sí a través de una red.

En este tipo de arquitectura, cada servicio representa una funcionalidad específica del sistema, como la gestión de usuarios, el procesamiento de pagos o el manejo de inventarios. Estos servicios pueden ser reutilizados por diferentes aplicaciones, lo que facilita la integración entre sistemas y mejora la flexibilidad del software.

La arquitectura orientada a servicios surge como una solución para integrar sistemas empresariales complejos, ya que permite que diferentes aplicaciones, incluso desarrolladas con tecnologías distintas, se comuniquen entre sí mediante interfaces estandarizadas.

Este enfoque presenta diversas características que facilitan el desarrollo, la integración y el mantenimiento de los sistemas:

- **Modularidad:** el sistema se divide en servicios independientes.
- **Reutilización:** un mismo servicio puede ser utilizado por diferentes aplicaciones.
- **Interoperabilidad:** permite la comunicación entre sistemas desarrollados en distintas tecnologías.
- **Escalabilidad:** los servicios pueden ampliarse o modificarse sin afectar todo el sistema.

- **Desacoplamiento:** cada servicio funciona de manera relativamente independiente.

La arquitectura orientada a servicios se fundamenta además en un conjunto de elementos que permiten estructurar y gestionar la comunicación entre los diferentes sistemas y aplicaciones. Estos elementos facilitan la publicación, el descubrimiento, el consumo y el intercambio de servicios dentro de una organización, garantizando interoperabilidad, reutilización y escalabilidad en los sistemas de información. A continuación, se presentan algunos de los componentes más representativos que hacen posible el funcionamiento de esta arquitectura.

Tabla 14. Componentes de la arquitectura orientada a servicios

Componente	Descripción	Ejemplo
Servicio.	Funcionalidad específica que el sistema ofrece.	Servicio de autenticación de usuarios.
Proveedor de servicio.	Sistema o aplicación que ofrece el servicio.	Servidor que gestiona el login.
Consumidor de servicio.	Aplicación que utiliza el servicio.	Aplicación web o móvil.
Contrato de servicio.	Define cómo se utiliza el servicio y qué información intercambia.	API o especificación de comunicación.

A continuación, un ejemplo práctico:

Supóngase una plataforma de comercio electrónico. En este sistema, diferentes funcionalidades pueden implementarse como servicios independientes:

- Servicio de gestión de usuarios.

- Servicio de procesamiento de pagos.
- Servicio de gestión de productos.
- Servicio de envíos y logística.

Cada uno de estos servicios puede funcionar de manera independiente y ser utilizado por diferentes aplicaciones, como la página web de la tienda, una aplicación móvil o un sistema interno de administración.

La arquitectura orientada a servicios ha sido un paso importante en la evolución de los sistemas distribuidos, ya que permite construir aplicaciones más flexibles, escalables y fáciles de integrar.

Además, este enfoque ha influido en arquitecturas modernas como los microservicios y el desarrollo basado en APIs, que continúan ampliando las posibilidades de integración entre sistemas en entornos tecnológicos actuales.

6.2. Arquitecturas basadas en APIs REST y GraphQL

Las arquitecturas basadas en APIs (Application Programming Interfaces) permiten que diferentes aplicaciones o sistemas se comuniquen entre sí mediante interfaces bien definidas. En este enfoque, los sistemas exponen servicios o recursos que pueden ser consumidos por otras aplicaciones a través de solicitudes realizadas generalmente mediante protocolos web.

Este modelo es ampliamente utilizado en aplicaciones web y móviles, ya que permite separar el frontend (interfaz de usuario) del backend (lógica del sistema y manejo de datos). De esta manera, diferentes aplicaciones pueden utilizar los mismos servicios sin necesidad de duplicar funcionalidades.

Entre los enfoques más utilizados para construir APIs se encuentran REST y GraphQL, los cuales ofrecen distintas formas de acceder y gestionar la información dentro de un sistema.

REST (Representational State Transfer) es un estilo arquitectónico que organiza los recursos del sistema mediante direcciones URL y utiliza métodos estándar del protocolo HTTP para interactuar con ellos. En este modelo, cada recurso puede ser consultado, creado, modificado o eliminado mediante solicitudes específicas.

Por su parte, GraphQL es un lenguaje de consulta para APIs que permite a los clientes solicitar exactamente la información que necesitan. A diferencia de REST, donde cada recurso suele tener un endpoint específico, GraphQL permite realizar consultas más flexibles y obtener múltiples datos en una sola solicitud.

Para comprender mejor las diferencias entre estos enfoques, se presenta la siguiente comparación:

Tabla 15. Comparación entre REST y GraphQL

Característica	REST	GraphQL
Modelo de acceso.	Basado en recursos.	Basado en consultas.
Endpoints.	Varios endpoints para diferentes recursos.	Un único endpoint.
Obtención de datos.	Devuelve estructuras definidas por el servidor.	El cliente solicita exactamente los datos necesarios.
Flexibilidad.	Menor flexibilidad en consultas.	Alta flexibilidad en consultas.
Uso común.	Servicios web tradicionales.	Aplicaciones modernas y aplicaciones móviles.

En muchos sistemas modernos, las APIs REST continúan siendo ampliamente utilizadas debido a su simplicidad y compatibilidad con tecnologías web. Sin embargo, GraphQL ha ganado popularidad en aplicaciones donde se requiere mayor flexibilidad en la consulta de datos y optimización en la comunicación entre cliente y servidor.

En síntesis, las arquitecturas basadas en APIs permiten construir sistemas más modulares, escalables e interoperables, facilitando la integración entre diferentes aplicaciones y servicios dentro de entornos tecnológicos modernos.

6.3. Arquitectura de microservicios

La arquitectura de microservicios es un enfoque de diseño de software en el que una aplicación se construye como un conjunto de servicios pequeños, independientes y especializados, donde cada uno cumple una función específica dentro del sistema.

A diferencia de las arquitecturas monolíticas, en las que toda la aplicación se desarrolla como una única unidad, la arquitectura de microservicios permite que cada componente se desarrolle, despliegue y escale de manera independiente. Esto facilita que los equipos de desarrollo trabajen de forma paralela y que el sistema evolucione con mayor flexibilidad.

En este modelo, cada microservicio se encarga de una responsabilidad específica del negocio, como la gestión de usuarios, el procesamiento de pagos o el manejo de productos dentro de una aplicación.

Este enfoque facilita la construcción de sistemas distribuidos que pueden adaptarse con mayor facilidad a cambios tecnológicos o a incrementos en la demanda de usuarios.

Tabla 16. Características de la arquitectura de microservicios

Característica	Descripción
Independencia.	Cada servicio puede desarrollarse y desplegarse de forma independiente.
Escalabilidad.	Es posible escalar únicamente los servicios que requieren mayor capacidad.
Especialización.	Cada microservicio se enfoca en una función específica del sistema.
Despliegue independiente.	Las actualizaciones pueden realizarse sin afectar todo el sistema.
Flexibilidad tecnológica.	Cada servicio puede desarrollarse utilizando diferentes tecnologías.

A continuación, un ejemplo conceptual:

Supóngase una plataforma de comercio electrónico. En una arquitectura de microservicios, el sistema podría dividirse en varios servicios independientes, tales como:

- Servicio de usuarios, encargado del registro y autenticación.
- Servicio de productos, responsable de gestionar el catálogo.
- Servicio de carrito de compras, que administra los productos seleccionados por el usuario.
- Servicio de pagos, encargado de procesar las transacciones.
- Servicio de envíos, responsable de gestionar la logística de entrega.

Cada uno de estos servicios puede ejecutarse de forma independiente y comunicarse con los demás mediante APIs.

Gracias a esta organización, si el sistema necesita procesar un mayor número de pagos, únicamente se puede escalar el servicio de pagos sin afectar el resto de los componentes.

En síntesis, la arquitectura de microservicios permite construir sistemas más modulares, escalables y adaptables, facilitando el mantenimiento, la evolución del software y el trabajo colaborativo en equipos de desarrollo.

6.4. Arquitectura serverless

La arquitectura serverless es un modelo de desarrollo en el que las aplicaciones se ejecutan sin que los desarrolladores tengan que administrar directamente los servidores o la infraestructura donde se ejecuta el software. Aunque los servidores siguen existiendo, su gestión es responsabilidad del proveedor de servicios en la nube.

En este modelo, el desarrollador se enfoca principalmente en escribir el código y definir las funciones que ejecutará el sistema, mientras que la plataforma se encarga automáticamente de tareas como el aprovisionamiento de recursos, la escalabilidad, la disponibilidad y el mantenimiento de la infraestructura.

La arquitectura serverless se basa en el concepto de ejecución bajo demanda, lo que significa que las funciones del sistema se ejecutan únicamente cuando ocurre un

evento específico, como una solicitud de un usuario, una actualización en una base de datos o la carga de un archivo.

Este enfoque permite optimizar el uso de los recursos y reducir costos, ya que el sistema solo consume infraestructura cuando realmente se utiliza.

Tabla 17. Características de la arquitectura serverless

Característica	Descripción	Beneficio
Gestión automática de infraestructura.	El proveedor en la nube administra servidores, redes y escalabilidad.	Reduce la complejidad operativa.
Ejecución basada en eventos.	Las funciones se ejecutan cuando ocurre una acción específica.	Uso eficiente de recursos.
Escalabilidad automática.	El sistema ajusta automáticamente la cantidad de recursos necesarios.	Soporta variaciones de carga sin intervención manual.
Pago por uso.	Solo se paga por el tiempo real de ejecución del código.	Optimización de costos.
Despliegue rápido.	Las funciones pueden actualizarse de manera independiente.	Mayor agilidad en el desarrollo.

A continuación, un ejemplo práctico:

Supóngase una aplicación web que permite a los usuarios subir fotografías a una plataforma en línea.

Cuando el usuario carga una imagen, se activa automáticamente una función serverless que realiza varias tareas:

- a) Verifica el formato y tamaño del archivo.
- b) Genera una versión comprimida de la imagen.

- c) Guarda la imagen en el almacenamiento del sistema.
- d) Actualiza la base de datos con la información del archivo.

En este proceso no es necesario mantener servidores activos de forma permanente. La función se ejecuta únicamente cuando ocurre la carga de la imagen.

En síntesis, la arquitectura serverless permite desarrollar aplicaciones más flexibles y escalables, delegando la gestión de la infraestructura a los proveedores de servicios en la nube y permitiendo que los desarrolladores se concentren principalmente en la lógica del negocio y en la creación de funcionalidades del sistema.

6.5. DevOps e integración continua

En el desarrollo moderno de software, los equipos buscan entregar aplicaciones de manera más rápida, confiable y continua. Para lograrlo, surge el enfoque DevOps, una práctica que integra el trabajo de los equipos de desarrollo (development) y operaciones (operations), promoviendo la colaboración, la automatización y la mejora continua en el ciclo de vida del software.

Tradicionalmente, el desarrollo y la operación de sistemas funcionaban como áreas separadas. Los desarrolladores creaban el software y posteriormente lo entregaban al equipo de operaciones para su implementación y mantenimiento. Este proceso generaba retrasos, dificultades en la comunicación y problemas al momento de desplegar nuevas versiones.

DevOps propone un enfoque colaborativo donde ambos equipos trabajan de forma coordinada durante todo el proceso de desarrollo, desde la programación hasta la implementación y monitoreo del sistema.

Una de las prácticas más importantes dentro de DevOps es la integración continua.

La integración continua consiste en integrar de manera frecuente los cambios realizados en el código dentro de un repositorio central. Cada vez que se realiza una modificación, el sistema ejecuta automáticamente procesos de compilación, pruebas y verificación para asegurar que el software funcione correctamente.

Tabla 18. Principales prácticas de DevOps

Práctica	Descripción	Beneficio
Integración continua.	Los cambios en el código se integran frecuentemente en un repositorio común.	Permite detectar errores rápidamente.
Entrega continua.	El software puede prepararse automáticamente para su publicación.	Reduce el tiempo de despliegue.
Automatización.	Se automatizan procesos como pruebas, compilación y despliegue.	Mejora la eficiencia del desarrollo.
Monitoreo continuo.	Se supervisa el comportamiento del sistema en producción.	Permite identificar fallos y mejorar el rendimiento.
Colaboración entre equipos.	Desarrollo y operaciones trabajan de manera integrada.	Mejora la comunicación y la calidad del software.

A continuación, un ejemplo práctico:

Supóngase que un equipo de desarrollo trabaja en una aplicación web:

- a) Un desarrollador realiza una mejora en el código.
- b) El cambio se envía a un repositorio de control de versiones.

- c) El sistema de integración continua detecta la actualización.
- d) Automáticamente se ejecutan pruebas para verificar que el sistema funcione correctamente.
- e) Si las pruebas son exitosas, la nueva versión puede prepararse para su despliegue.

Este proceso permite detectar errores de forma temprana y mantener el software actualizado de manera constante.

En síntesis, DevOps y la integración continua permiten mejorar la calidad del software, reducir los tiempos de desarrollo y facilitar la entrega continua de nuevas funcionalidades, contribuyendo a la creación de sistemas más confiables y eficientes.

7. Plataformas tecnológicas y documentación del diseño

En el desarrollo de software, la selección de las plataformas tecnológicas y la elaboración de la documentación de diseño constituyen actividades fundamentales para garantizar la calidad, organización y sostenibilidad de los sistemas. Las plataformas tecnológicas proporcionan las herramientas, lenguajes, frameworks y entornos necesarios para construir y ejecutar las aplicaciones, mientras que la documentación permite registrar las decisiones técnicas tomadas durante el proceso de diseño.

La adecuada elección de tecnologías influye directamente en aspectos como el rendimiento, la escalabilidad, la seguridad y la facilidad de mantenimiento del sistema. Por esta razón, los equipos de desarrollo deben analizar diferentes alternativas tecnológicas antes de iniciar la implementación.

Por otra parte, la documentación del diseño cumple un papel esencial dentro del ciclo de vida del software, ya que permite comunicar la estructura del sistema, las decisiones arquitectónicas y la organización de los componentes a todos los integrantes del equipo de desarrollo.

En conjunto, las plataformas tecnológicas y la documentación del diseño permiten estructurar el desarrollo del sistema de manera organizada, facilitando la colaboración entre los miembros del equipo y garantizando que las decisiones técnicas queden registradas para futuras etapas del proyecto.

7.1. Plataformas tecnológicas para el desarrollo de software

Las plataformas tecnológicas para el desarrollo de software corresponden al conjunto de herramientas, lenguajes, frameworks y entornos que permiten diseñar,

construir, ejecutar y mantener aplicaciones informáticas. Estas plataformas proporcionan la infraestructura necesaria para que los desarrolladores puedan implementar las funcionalidades del sistema y gestionar los diferentes componentes que lo conforman.

La elección de una plataforma tecnológica depende de diversos factores, entre ellos el tipo de aplicación que se desea desarrollar, los requerimientos del sistema, el entorno de ejecución y las necesidades de escalabilidad o mantenimiento. Una selección adecuada de tecnologías contribuye a mejorar el rendimiento del sistema, facilitar su evolución y garantizar la compatibilidad con otros sistemas.

En el desarrollo de software, las plataformas tecnológicas suelen integrarse en un ecosistema que incluye herramientas de programación, entornos de desarrollo, motores de bases de datos y sistemas de gestión de versiones.

Tabla 19. Componentes de una plataforma tecnológica de desarrollo

Componente	Descripción	Ejemplo
Lenguajes de programación.	Permiten escribir el código fuente del sistema y definir su lógica de funcionamiento.	Java, Python y JavaScript.
Frameworks.	Conjunto de herramientas y librerías que facilitan el desarrollo de aplicaciones.	Spring, Django y React.
Entornos de desarrollo (IDE).	Aplicaciones que proporcionan herramientas para escribir, probar y depurar código.	IntelliJ IDEA y Visual Studio Code.
Motores de bases de datos.	Sistemas que permiten almacenar, gestionar y consultar información.	MySQL, PostgreSQL y MongoDB.

Componente	Descripción	Ejemplo
Sistemas de control de versiones.	Herramientas que permiten gestionar cambios en el código fuente.	Git.

La integración de estos componentes permite construir entornos de desarrollo completos que facilitan la creación de aplicaciones robustas y escalables. Además, el uso adecuado de plataformas tecnológicas contribuye a mejorar la productividad de los equipos de desarrollo y a mantener la organización del proyecto a lo largo de su ciclo de vida.

7.2. Motores de bases de datos relacionales y no relacionales

Los motores de bases de datos son sistemas encargados de almacenar, gestionar y recuperar la información utilizada por las aplicaciones de software. Estos motores permiten organizar grandes volúmenes de datos de manera estructurada y facilitan su consulta, actualización y mantenimiento.

En el desarrollo de software, la selección del motor de base de datos depende de diversos factores, como el tipo de aplicación, la cantidad de información que se debe manejar, el nivel de concurrencia de usuarios y los requerimientos de escalabilidad del sistema.

De manera general, los motores de bases de datos pueden clasificarse en dos grandes categorías: relacionales y no relacionales.

Tabla 20. Tipos de motores de bases de datos

Tipo de base de datos	Características principales	Ejemplos
Bases de datos relacionales.	Organizan la información en tablas compuestas por filas y columnas. Utilizan un esquema estructurado y permiten establecer relaciones entre las diferentes tablas mediante claves.	MySQL, PostgreSQL y Oracle.
Bases de datos no relacionales.	Almacenan la información utilizando estructuras más flexibles, como documentos, grafos o pares clave-valor. Son utilizadas en aplicaciones que requieren alta escalabilidad y manejo de grandes volúmenes de datos.	MongoDB, Cassandra y Redis.

Las bases de datos relacionales han sido ampliamente utilizadas en sistemas empresariales debido a su capacidad para mantener la integridad de los datos y establecer relaciones claras entre la información almacenada. Estas bases utilizan el lenguaje SQL para realizar consultas, inserciones, actualizaciones y eliminación de datos.

Por otra parte, las bases de datos no relacionales, también conocidas como NoSQL, han surgido como una alternativa para aplicaciones modernas que manejan grandes volúmenes de información, como plataformas web, redes sociales o sistemas de análisis de datos.

En síntesis, la elección entre un motor de base de datos relacional o no relacional depende de las necesidades específicas del sistema. Mientras los motores relacionales ofrecen una estructura organizada y consistente para la gestión de datos, los motores

no relacionales brindan mayor flexibilidad y escalabilidad para aplicaciones distribuidas o con altos volúmenes de información.

Para complementar la información sobre bases de datos relacionales y no relacionales, se recomienda acceder al siguiente video:

<https://www.youtube.com/watch?v=r97Ko4ZvIDQ>

7.3. Documento de diseño de software

El documento de diseño de software es un artefacto técnico que describe la estructura, organización y funcionamiento del sistema que se desea desarrollar. Este documento permite registrar de manera formal las decisiones de diseño tomadas durante el proceso de desarrollo y sirve como guía para los equipos encargados de implementar la solución.

En el proceso de ingeniería de software, el documento de diseño cumple un papel fundamental, ya que traduce los requerimientos del sistema en una propuesta estructurada que describe cómo se construirá la aplicación. De esta manera, facilita la comunicación entre analistas, arquitectos de software, desarrolladores y demás actores involucrados en el proyecto.

Además, este documento permite mantener una referencia clara sobre la arquitectura del sistema, los componentes que lo conforman, las tecnologías seleccionadas y las relaciones entre los distintos módulos del software.

Un documento de diseño de software suele incluir diferentes secciones que describen aspectos técnicos y estructurales del sistema.

Tabla 21. Componentes de un documento de diseño de software

Componente	Descripción	Propósito
Descripción general del sistema.	Presenta una visión general del sistema, sus objetivos y alcance.	Contextualizar el sistema dentro del proyecto.
Arquitectura del sistema.	Describe la estructura general del software y el modelo arquitectónico utilizado.	Definir la organización del sistema.
Componentes del sistema.	Identifica los módulos o componentes que conforman la aplicación.	Explicar cómo se divide el sistema.
Modelo de datos.	Describe la estructura de los datos y su organización en la base de datos.	Definir cómo se almacenará la información.
Diagramas del sistema.	Incluye diagramas como diagramas de clases, componentes o despliegue.	Facilitar la comprensión visual del sistema.
Tecnologías utilizadas.	Indica los lenguajes, frameworks y herramientas empleadas.	Documentar la infraestructura tecnológica del proyecto.

La elaboración de este documento permite planificar de manera organizada el desarrollo del software, reducir riesgos durante la implementación y facilitar el mantenimiento del sistema en el futuro.

Asimismo, el documento de diseño sirve como base para la documentación técnica del proyecto, permitiendo que nuevos miembros del equipo comprendan rápidamente la estructura y funcionamiento del sistema.

En síntesis, el documento de diseño de software constituye una herramienta esencial dentro del proceso de desarrollo, ya que permite documentar la arquitectura,

los componentes y las decisiones técnicas del sistema, garantizando una construcción más organizada, comprensible y sostenible del software.

7.4. Arquitectura candidata

La arquitectura candidata corresponde a la propuesta inicial de arquitectura de software que se plantea para satisfacer los requerimientos funcionales y no funcionales de un sistema. Esta arquitectura se construye a partir del análisis de los requerimientos, las restricciones tecnológicas, las necesidades del negocio y las características del entorno donde será implementado el sistema.

Durante el proceso de diseño de software, la arquitectura candidata permite definir una estructura preliminar del sistema, identificando los principales componentes, las tecnologías que se utilizarán y la forma en que estos elementos interactúan entre sí. Esta propuesta sirve como base para orientar el desarrollo del sistema y facilitar la toma de decisiones técnicas.

La arquitectura candidata no representa necesariamente la versión final del diseño del sistema. En muchos casos, puede ser ajustada o refinada a medida que el equipo de desarrollo profundiza en el análisis del proyecto o surgen nuevos requerimientos.

Para construir una arquitectura candidata se consideran diversos elementos que permiten estructurar adecuadamente el sistema:

Requerimientos del sistema

Corresponden a las necesidades funcionales y no funcionales que el software debe cumplir. Estos requerimientos orientan las decisiones de diseño y garantizan que la arquitectura responda a los objetivos del sistema.

Modelo arquitectónico

Define el tipo de arquitectura que se utilizará, como cliente-servidor, microservicios o modelo vista controlador. Este modelo establece la forma en que se organizarán los componentes del sistema.

Componentes del sistema

Se refiere a los módulos, servicios o capas que conforman el software. Identificar estos componentes permite estructurar la lógica del sistema y facilitar su desarrollo.

Tecnologías y plataformas

Incluye los lenguajes de programación, frameworks, bases de datos y herramientas tecnológicas que se utilizarán en el proyecto.

Interacción entre componentes

Describe la forma en que los diferentes módulos del sistema se comunican entre sí, permitiendo el intercambio de información y el funcionamiento integrado del software.

La definición de una arquitectura candidata permite evaluar distintas alternativas de diseño antes de iniciar la implementación del sistema. Esto ayuda a seleccionar la

arquitectura más adecuada de acuerdo con criterios como escalabilidad, rendimiento, seguridad y facilidad de mantenimiento.

Asimismo, la arquitectura candidata facilita la comunicación entre los miembros del equipo de desarrollo, ya que proporciona una representación clara de cómo se organizará el sistema y cómo se integrarán sus diferentes componentes.

A continuación, un ejemplo conceptual de arquitectura candidata:

Supóngase que una empresa desea desarrollar una aplicación web para gestionar reservas de hoteles. Antes de iniciar la programación, el equipo de desarrollo propone una arquitectura candidata que permita organizar el sistema.

En esta propuesta inicial se definen los siguientes elementos:

- **Interfaz de usuario:** se desarrollará una aplicación web a la que los usuarios accederán desde su navegador para buscar hoteles, consultar disponibilidad y realizar reservas.
- **Servidor de aplicación:** el servidor procesará las solicitudes enviadas por los usuarios, gestionará la lógica del sistema y controlará las operaciones relacionadas con las reservas.
- **Base de datos:** se utilizará un sistema gestor de bases de datos para almacenar información sobre hoteles, habitaciones, usuarios y reservas.
- **Servicios del sistema:** se implementarán servicios que permitirán gestionar funcionalidades como autenticación de usuarios, consulta de disponibilidad y registro de reservas.

De manera simplificada, la arquitectura candidata podría representarse así:

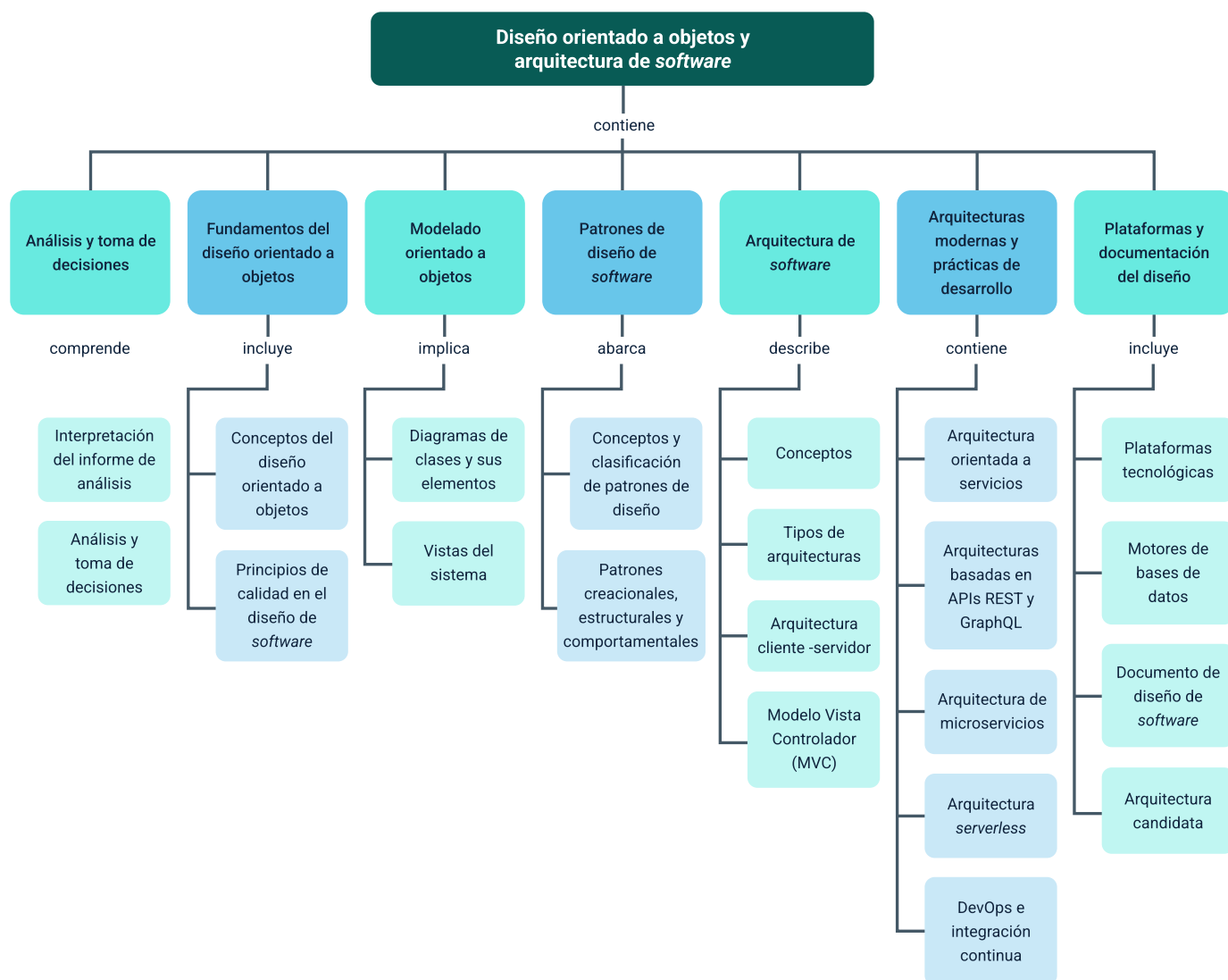
Usuario (navegador web) - Aplicación web - Servidor de aplicación - Base de datos

Esta propuesta permite al equipo de desarrollo analizar si la arquitectura seleccionada responde a los requerimientos del sistema antes de iniciar su implementación.

En síntesis, la arquitectura candidata constituye una propuesta inicial de diseño que orienta el desarrollo del software, permitiendo estructurar el sistema de manera coherente y asegurar que las decisiones técnicas respondan a los objetivos del proyecto.

Síntesis

El componente formativo aborda los fundamentos y procesos esenciales para el diseño de software, iniciando con el análisis de requerimientos y la interpretación del informe de análisis como base para la toma de decisiones de diseño. Posteriormente, se presentan los principios del diseño orientado a objetos y los criterios de calidad que orientan la construcción de soluciones estructuradas y mantenibles. A partir de estos fundamentos, se estudian herramientas de modelado como los diagramas de clases y las vistas del sistema, que permiten representar la estructura y organización de una aplicación. Asimismo, se analizan los patrones de diseño y su clasificación, junto con los principales enfoques de arquitectura de software, incluyendo modelos como cliente-servidor y Modelo Vista Controlador (MVC). Finalmente, se examinan arquitecturas modernas y prácticas actuales de desarrollo, como servicios, APIs, microservicios, serverless y DevOps, además de las plataformas tecnológicas, los motores de bases de datos y los documentos de diseño que permiten definir y documentar la arquitectura candidata de un sistema.



Glosario

API (Interfaz de Programación de Aplicaciones): conjunto de reglas y mecanismos que permite que diferentes aplicaciones o sistemas se comuniquen entre sí e intercambien información o funcionalidades.

Arquitectura de software: estructura fundamental de un sistema de software que define la organización de sus componentes, sus relaciones y los principios que guían su diseño y evolución.

Arquitectura cliente-servidor: modelo de arquitectura en el que los clientes solicitan servicios o recursos y los servidores procesan esas solicitudes y envían las respuestas correspondientes.

Arquitectura orientada a servicios (SOA): enfoque arquitectónico basado en la creación de servicios reutilizables que pueden ser utilizados por diferentes aplicaciones dentro de un sistema o entre varios sistemas.

DevOps: conjunto de prácticas que integran el desarrollo de software y las operaciones de tecnología para mejorar la colaboración, automatizar procesos y acelerar la entrega de aplicaciones.

Diagrama de clases: representación gráfica utilizada en el modelado orientado a objetos que presenta las clases de un sistema, sus atributos, métodos y relaciones.

GraphQL: lenguaje de consulta para APIs que permite a los clientes solicitar exactamente los datos que necesitan, optimizando la comunicación entre cliente y servidor.

Modelo Vista Controlador (MVC): patrón arquitectónico que separa una aplicación en tres componentes: modelo (gestión de datos), vista (interfaz de usuario) y controlador (lógica que conecta ambos).

Plataforma tecnológica: conjunto de herramientas, lenguajes, frameworks y recursos tecnológicos utilizados para desarrollar, ejecutar y mantener aplicaciones de software.

REST (Representational State Transfer): estilo arquitectónico utilizado para diseñar servicios web que se comunican mediante protocolos estándar de Internet, generalmente utilizando HTTP.

Referencias bibliográficas

Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.

Ecosistema de Recursos Educativos Digitales SENA. (2021). Bases de datos relacionales y no relacionales [Video]. YouTube.

<https://www.youtube.com/watch?v=r97Ko4ZvIDQ>

Ecosistema de Recursos Educativos Digitales SENA. (2021). Patrones creacionales [Video]. YouTube. <https://www.youtube.com/watch?v= KZkbL0MMbQ>

Ecosistema de Recursos Educativos Digitales SENA. (2024). Componentes de una arquitectura de software [Video]. YouTube.

<https://www.youtube.com/watch?v=kiV7aeJCqUQ>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Shaw, M., & Garlan, D. (1996). Software architecture: Perspectives on an emerging discipline. Prentice Hall.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluada instrucción	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluada de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima